



Module 14:

Algorithm

Complexity

CPE010 DATA STRUCTURES AND ALGORITHMS

Analyzing the Complexity of Algorithms through Asymptotic Analysis

Intended Learning Outcomes

After this module, the student should be able to:

- Analyze given algorithms to determine space and time complexity.
- Compute for the space and time complexity of a given algorithm.

Asymptotic Analysis

- To compare two algorithms with running times $f(n)$ and $g(n)$, we need a **rough measure** that characterizes **how fast each function grows**.
- **Hint:** use rate of growth
- Compare functions in the limit, that is, asymptotically!
 - (i.e., for large values of n)

Growth Rate

- Consider the example of buying *elephants* and *goldfish*:
 - **Cost**: cost_of_elephants + cost_of_goldfish
 - **Cost** ~ cost_of_elephants (approximation)
- The low order terms in a function are relatively insignificant for **large** n

$$n^4 + 100n^2 + 10n + 50 \approx n^4$$

We say that $n^4 + 100n^2 + 10n + 50$ and n^4 have the same **rate of growth**.

Asymptotic Notation

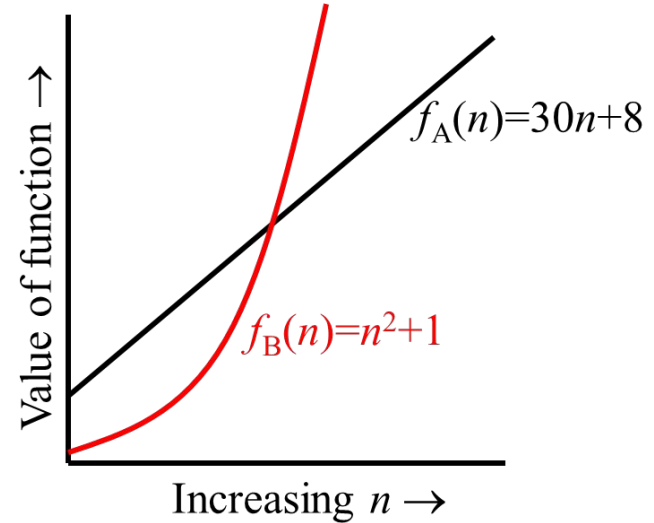
Asymptotic Notation is used to describe the running time of an algorithm - **how much time an algorithm takes with a given input, n** . There are three different notations: big O, big Theta (Θ), and big Omega (Ω).

- O notation: asymptotic “less than”
 - $f(n) = O(g(n))$ implies that $f(n) \leq g(n)$
- Ω notation: asymptotic “greater than”
 - $f(n) = \Omega(g(n))$ implies that $f(n) \geq g(n)$
- Θ notation: asymptotic “equality”
 - $f(n) = \Theta(g(n))$ implies that $f(n) = g(n)$

Big-O Notation

- $f_A(n)=30n+8$ is order n , or $O(n)$.
 - It is, at most, roughly proportional to n .
- $f_B(n)=n^2+1$ is order n^2 , or $O(n^2)$.
 - It is, at most, roughly proportional to n^2 .

In general, any $O(n^2)$ function is faster-growing than any $O(n)$ function.



On a graph, as you go to the right, a faster growing function eventually becomes larger.

Example 1 - Big-O Notation

- $n^4 + 100n^2 + 10n + 50$ is **$O(n^4)$**
- $10n^3 + 2n^2$ is **$O(n^3)$**
- $n^3 - n^2$ is **$O(n^3)$**

Constants:

- 10 is $O(1)$
- 1273 is $O(1)$

Example 2 - Big-O Notation

What is the order of both algorithms shown below?

Algorithm 1	Algorithm 2
<pre>arr[0] = 0; arr[1] = 0; arr[2] = 0; ... arr[n-1] = 0</pre>	<pre>for(i = 0; i < N; i++) arr[i] = 0;</pre>

Answer in chat.

Example 2 - Big-O Notation

What is the order of both algorithms shown below?

Algorithm 1	Algorithm 2
<pre>arr[0] = 0; arr[1] = 0; arr[2] = 0; ... arr[n-1] = 0</pre>	<pre>for(i = 0; i < N; i++) arr[i] = 0;</pre>

$O(n)$

Example 3 - Big-O Notation

```
sum = 0;  
for(i = 0; i < n; i++)  
    for(j = 0; j < n; j++)  
        Sum += arr[i][j]
```

What is the order of this algorithm shown?

Example 3 - Big-O Notation

```
sum = 0;  
for(i = 0; i < n; i++)  
    for(j = 0; j < n; j++)  
        Sum += arr[i][j]
```

What is the order of this algorithm shown?

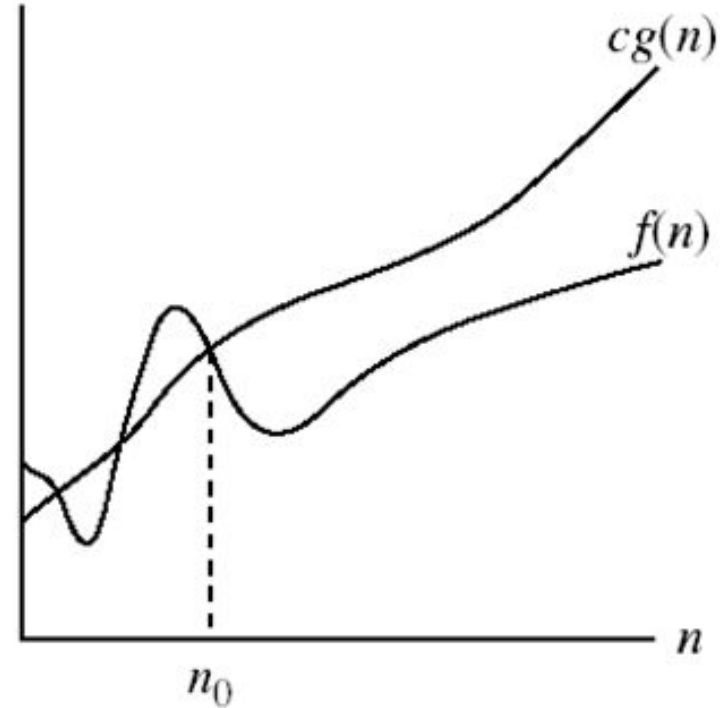
$O(n^2)$

Asymptotic Notations

$O(g(n)) = \{f(n) : \text{there exists positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}.$

This figure shows that $g(n)$ is an **asymptotic upper bound** for $f(n)$.

$O(g(n))$ is the **set of functions with smaller or same order** of growth as $g(n)$.



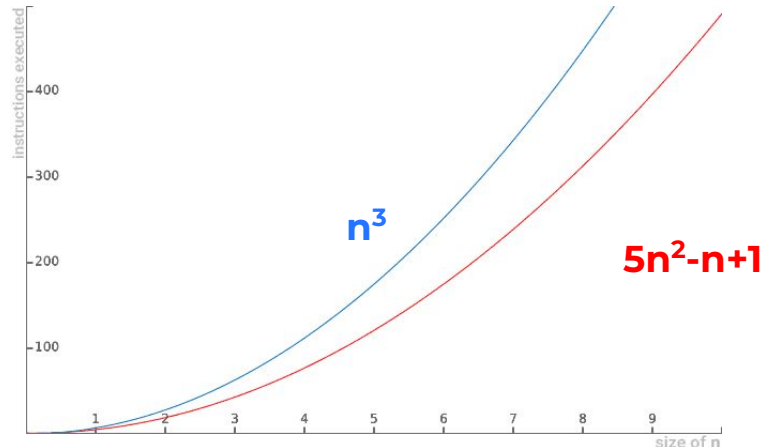
Example 4 - Big-O Demonstration

Proving that $5n^2 - n + 1$ is $O(n^2)$.

- This is meant to say that the asymptotic behavior of a quadratic function is **equal or greater** than its own.
- The expression will **never** grow past a quadratic behavior.
- $O(n^2)$ is meant to indicate that the function will never exhibit behavior that exceeds this upper limit.
- Let's check this graph:

We can also say that $5n^2 - n + 1$ is $O(n^3)$. Why?
Because it will never exceed cubic behavior.

Therefore, $T(n) = O(n^3)$.



Example 4 - Big-O Demonstration

Summary

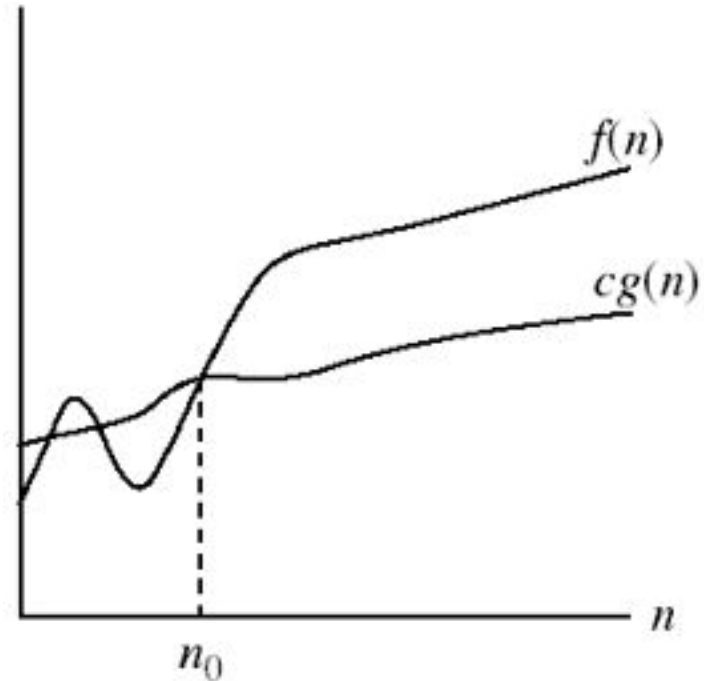
- $5n^2 - n + 1$ is $O(n^3)$
- $5n^2 - n + 1$ is $O(n^2)$
- $5n^2 - n + 1$ is not $O(n)$

Asymptotic Notations

Ω Big Omega Notation

$\Omega(g(n)) = \{ f(n) : \text{there exists positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0 \}$.

$\Omega(g(n))$ is the **set of functions with larger or same order of growth** as $g(n)$.

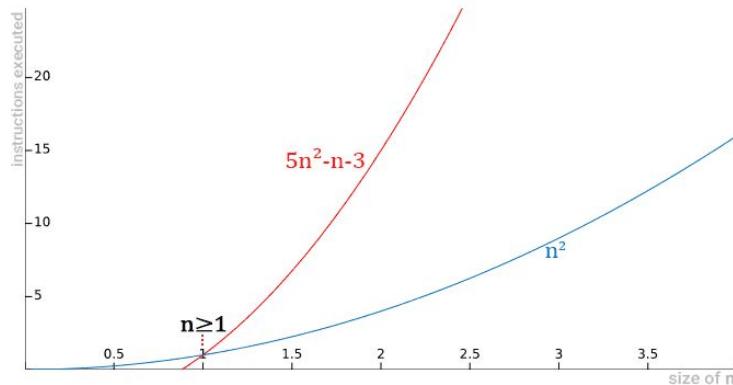


$g(n)$ is an **asymptotic lower bound** for $f(n)$.

Example 5 - Big-Omega Demonstration

Proving that $5n^2 - n - 3$ is $\Omega(n^2)$.

- This is meant to say that the asymptotic behavior of a quadratic function is **equal to or less than** its own.
- The function will never have a **lower** growth behavior than a function with quadratic behavior.
- $\Omega(n^2)$ indicates that the function will never exhibit a growth behavior that is exceeded by this lower limit.
- Let's check this graph:



This graph indicates that for values $n \geq 1$, our function outperforms function n^2 .

It is also correct to say $\Omega(n)$ because our function will never be surpassed by a linear function.

Therefore, $T(n) = \Omega(n)$.

Example 5 - Big-Omega Demonstration

Summary

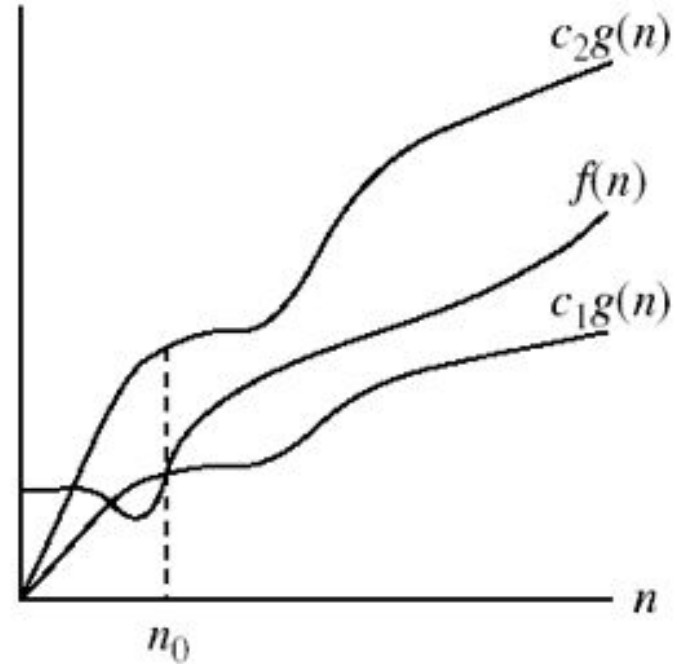
- $5n^2 - n - 3$ is not $\Omega(n^3)$.
- $5n^2 - n - 3$ is $\Omega(n^2)$.
- $5n^2 - n - 3$ is $\Omega(n)$.

Asymptotic Notations

Θ Big-Theta Notation

$\Theta(g(n)) = \{ f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0 \}.$

$\Theta(g(n))$ is the **set of functions with the same order of growth** as $g(n)$.



$g(n)$ is an **asymptotic tight bound** for $f(n)$.

Example 6 - Big-Theta Demonstration

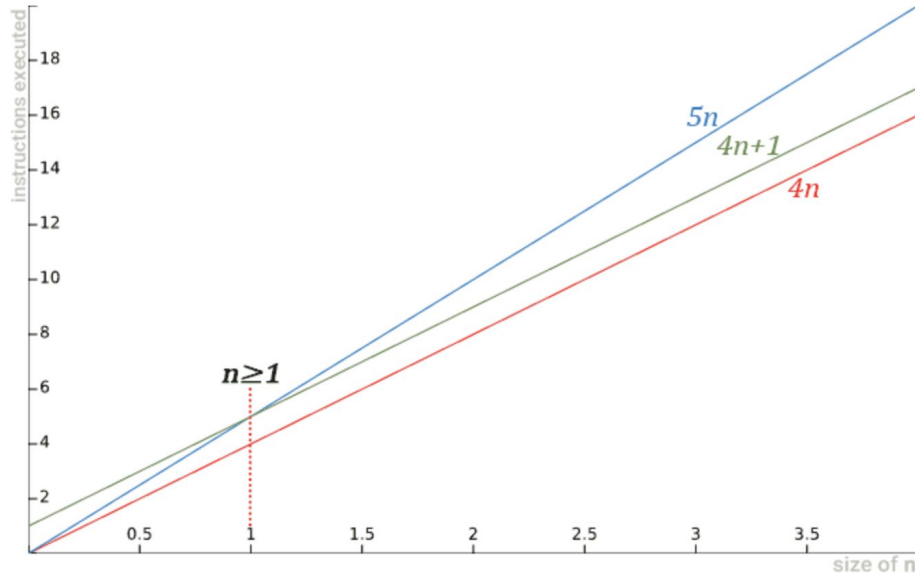
Let's check if it's true that $4n+1 = \Theta(n)$.

- All we have to do is prove that:
 - Two positive constants c_1 and c_2 , and a positive initial value n_0 so that for all values of $n \geq n_0$:
 - $4n+1$ is always **greater than or equal to** $c_1 * n$
 - $4n+1$ is always **less than or equal to** $c_2 * n$
 - Must be satisfied simultaneously.
 - Mathematically speaking: $c_1 n \leq 4n+1 \leq c_2 n$

Let's consider the graph if the values of c_1 and c_2 are 4 and 5; respectively.

Example 6 - Big-Theta Demonstration

Let's consider the graph if the values of c_1 and c_2 are 4 and 5; respectively.



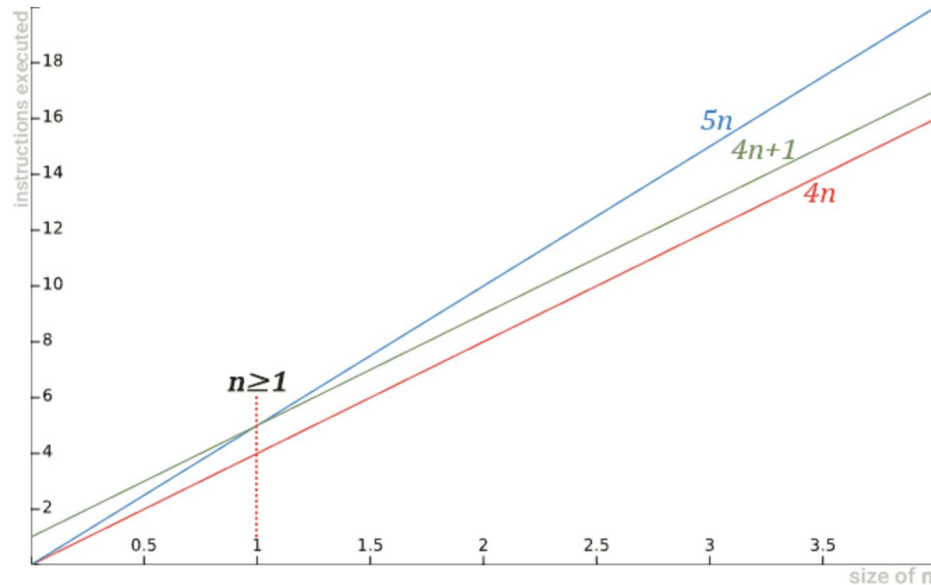
FILL OUT THE TABLE BELOW.

Provide your analysis!

n	4n	4n+1	5n
1			
2			
3			
4			
5			

Example 6 - Big-Theta Demonstration

Let's consider the graph if the values of c_1 and c_2 are 4 and 5; respectively.

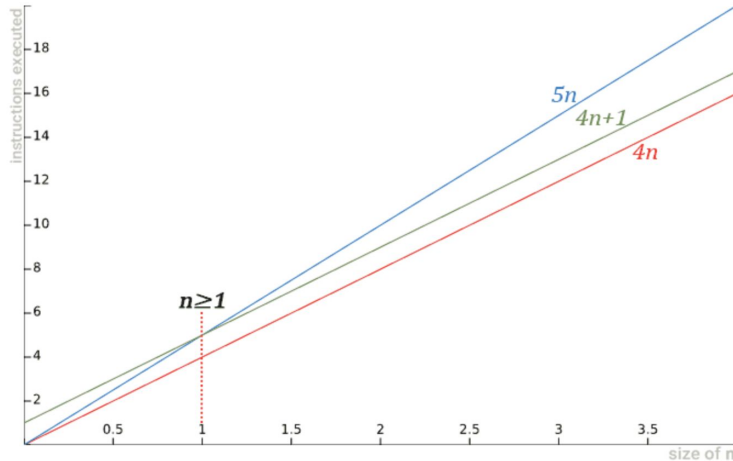


When we inspect the different values of n for the equation, we can observe that the function holds true.

n	4n	4n+1	5n
1	4	5	5
2	8	9	10
3	12	13	15
4	16	17	20
5	20	21	25

Example 6 - Big-Theta Demonstration

Let's consider the graph if the values of c_1 and c_2 are 4 and 5; respectively.



When we inspect the different values of n for the equation, we can observe that the function holds true.

n	4n	4n+1	5n
1	4	5	5
2	8	9	10
3	12	13	15
4	16	17	20
5	20	21	25

This means that the $4n+1$ function **will never grow below $4n$ and will not grow more than $5n$** . For this reason we say that **$\Theta(n)$ represents a fair asymptotic limit for the $4n+1$ function**, because it is limited upper and lower by two other functions of the same asymptotic class, the linear class.

Example 7 - Relationship of Asymptotic Notations

Let's look at an algorithm that is $T(n) = 4n+4$ and create an asymptotic relationship table.

	Big-O	Big-Omega	Big-Theta
$n!$			
2^n			
n^2			
$n \log n$			
n			
$\log n$			
1			

ANSWER ON YOUR OWN. *Answers are in the next slide!*

Example 7 - Relationship of Asymptotic Notations

Let's look at an algorithm that is $T(n) = 4n+4$ and create an asymptotic relationship table.

	Big-O	Big-Omega	Big-Theta
$n!$	Yes	No	No
2^n	Yes	No	No
n^2	Yes	No	No
$n \log n$	Yes	No	No
n	Yes	Yes	Yes
$\log n$	No	Yes	No
1	No	Yes	No

Compare to your answers; provide an analysis (unmute your mic)!

Example 8 - Bubble Sort Complexity

Let's take into account the best and worst case of the algorithm. We can define the Bubble Sort algorithm to have the following cases:

- W.C. : $T(n) = 5n^2 - n + 1$
- B.C. : $T(n) = 3n + 2$

How do we express this with our asymptotic notations:

- W.C. : $T(n) = \Theta(n^2)$
- B.C. : $T(n) = \Theta(n)$

BUT THE ASYMPTOTIC LIMITS are expressed as:

- $O(n^2)$
- $\Omega(n)$

Example 8 - Bubble Sort Complexity

Let's take into account the best and worst case of the algorithm. We can define the Bubble Sort algorithm to have the following cases:

- W.C. : $T(n) = 5n^2 - n + 1$
- B.C. : $T(n) = 3n + 2$

How do we express this with our asymptotic notation:

- W.C. : $T(n) = \Theta(n^2)$ *Theta of n squared*
- B.C. : $T(n) = \Theta(n)$ *Theta of n*

BUT THE ASYMPTOTIC LIMITS are expressed as:

- $O(n^2)$
- $\Omega(n)$

This is because our Bubble Sort behaves differently in the best and worst cases.

Concept Check:

Provide an analysis of merge sort vs. quick sort using asymptotic analysis; submit in the discussion post for this module.



Thanks for participating!

Submit your questions through the discussion
Q&A found on module 14.